

Celaya NFC 控制器 Linux 软件开发

移植指南

文档版本 05

发布日期 2024-10-16

前言

概述

本文档从软件开发角度说明了将 NFC（Near Field Communication）控制器适配到 Linux 系统的方法。主要包括 Linux NFC 软件总体框图、内核驱动移植方法、NFC 控制器业务代码与配置文件说明、常见问题等。

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

修改记录

文档版本	发布日期	修改说明
05	2024-10-16	更新“ 2.2 获取驱动源码 ”中的文件描述。
04	2023-06-27	新增“2.5-测试驱动”
03	2023-02-06	- 更新“4 NFC 参数配置文件”中的 SET_RF_PARAM 配置参数说明。 - 更新“2.3 代码移植和适配”中的驱动目录和文件描述等。 - 新增“2.3.4 安全函数库编译指导”和“2.3.5 DTS 配置”。

文档版本	发布日期	修改说明
02	2022-08-23	更新 “3.5 步骤五：NFCC 固件部署” 小节内容描述。
01	2022-06-23	第一次正式版本发布。

目 录

前言i

1 Linux NFC 系统架构.....1

2 NFC 内核驱动3

2.1 驱动描述..... 3

2.2 获取驱动源码 3

2.3 代码移植和适配..... 4

2.3.1 oal_os.h OS 接口 5

2.3.2 oal_plat_op.h 文件操作和日志打印接口 6

2.3.3 board_q.c 板级接口 6

2.3.4 安全函数库编译指导 7

2.3.5 DTS 配置 9

2.4 编译内核..... 10

2.5 测试驱动..... 10

3 Linux NFC 业务代码.....11

3.1 步骤一：安装 NFC 源码包11

3.2 步骤二：编译 NFC HAL 库文件 12

3.3 步骤三：集成 NFC 协议栈构建应用..... 12

3.4 步骤四：更新 NFC 参数配置文件 12

3.5 步骤五：NFCC 固件部署 13

3.6 步骤六：测试 NFC 功能..... 13

4 NFC 参数配置文件14

5 常见问题.....15

5.1 驱动打开失败	15
5.2 配置文件打开失败	15
5.3 共享库打开失败	16
A 缩略语	17

插图目录

图 1-1 Linux NFC 系统架构 1

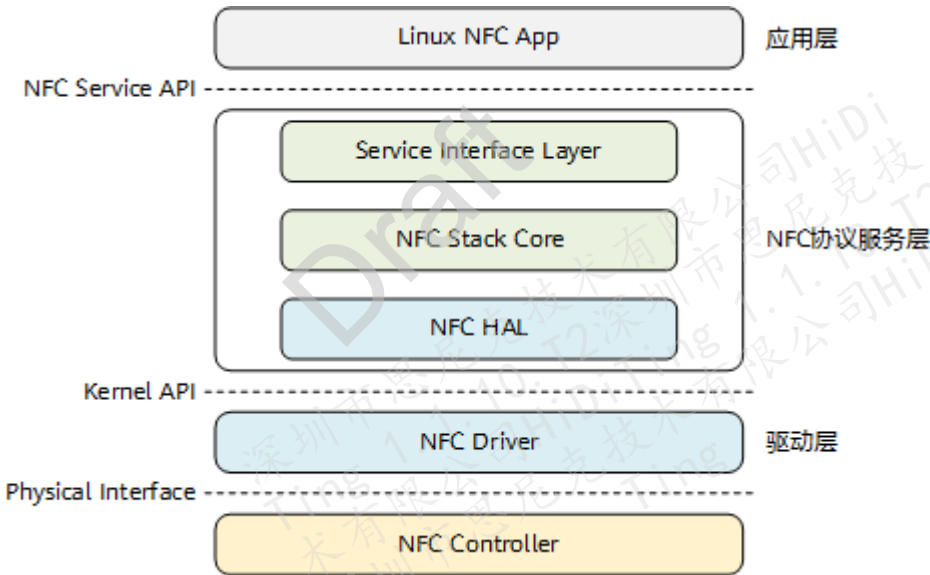
表格目录

表 2-1 驱动代码说明	4
表 2-2 oal_os.h 接口函数说明	5
表 2-3 oal_plat_op.h 接口函数说明	6
表 2-4 board_q.c 接口函数说明	7
表 4-1 libnfc-hal.conf 文件配置参数表	14

1 Linux NFC 系统架构

Linux NFC 系统架构如图 1-1 所示。

图1-1 Linux NFC 系统架构



Linux NFC 系统架构图说明如下：

- 框图颜色说明：灰色为应用层用户软件，绿色为开源软件或用户自有软件，蓝色为本方案软件，橙色为本方案硬件。
- NFC Driver 是 NFC 的内核驱动，用于访问底层 NFC 控制器资源，相关移植细节请参见“2 NFC 内核驱动”章节。
- NFC HAL 是硬件抽象层，用于向上层提供硬件服务的抽象接口，是“3 Linux NFC 业务代码”章节主要的软件移植内容。

- NFC Stack Core 是 NCI (NFC Controller Interface) 协议栈, 完成基于 NCI 的业务状态机控制。
- NFC Service Interface Layer 作为框架服务层的顶层, 对上层 App 提供接口服务。
- Linux NFC App 是应用层用户软件, 提供 NFC 服务。

Draft

2 NFC 内核驱动

- 2.1 驱动描述
- 2.2 获取驱动源码
- 2.3 代码移植和适配
- 2.4 编译内核
- 2.5 测试驱动

2.1 驱动描述

NFC 驱动通过 I2C 和底层的 NFC 控制器通讯，当驱动加载起来后，会提供给上层一个访问节点/dev/nfc_dev。集成商修改该驱动节点名称，同时需要修改协议栈配置文件“libnfc-hal.conf”中的 NFC_DEV_NODE 参数值（详细内容请参见“[3 Linux NFC 业务代码](#)”章节）。

2.2 获取驱动源码

获取 NFC 驱动源码包，解压到 kernel 目录下的 drivers/nfc 路径下（原路径下部分文件会被覆盖，请提前备份）。解压后 Vendor NFC 驱动相关文件说明如下：

- Makefile：驱动代码的 makefile。
- Kconfig：驱动配置参数。
- comm_driver：驱动通用代码的子文件夹，具体文件请见源码包。
- adapter：具体产品下驱动源码的子文件夹，具体文件请见源码包。

获取依赖的第三方库为“libboundscheck”，获取链接为<https://gitee.com/openeuler/libboundscheck>。此库主要提供 memcpy、strcpy 等内存操作库函数的安全版本。NFC 驱动代码使用了安全版本的内存操作函数。

2.3 代码移植和适配

NFC 驱动需要通过简单的移植和适配，嵌入 kernel 环境才能编译运行。如“2.2 获取驱动源码”小节介绍，驱动代码主要分为 comm_driver 模块和 adapter 模块，代码分别放在对应的目录。

- comm_driver 实现主要的驱动功能，不需要改动。
- adapter 需要用户适配板级接口，这些接口由 comm_driver 调用。adapter 中提供了/dev/nfc_dev 文件节点，作为用户态的接口。

代码结构与详细描述如表 2-1 所示。

表2-1 驱动代码说明

目录	目录下的子文件夹和文件说明		
adapter(dir)	需要用户根据应用环境适配，实现固定的系统和板级接口。		
	oal(dir)	系统抽象层，适配不同 OS 接口。	
	-	linux(dir)	linux 系统，unix-like 已适配，用户不需要修改。
		oal_os.h	os 接口适配。
		oal_plat_op.h	文件操作接口和日志打印接口适配。
	product(dir)	产品板级接口适配。	
	-	phone(dir)	Android 和 Linux 环境适用
		board_q.c	实现 board_nfc.h 定义的板级接口，这些接口被 comm_driver 模块调用，需要用户结合不同设备环境进行适配。

目录	目录下的子文件夹和文件说明			
			nfc_dev_q.h nfc_dev_q.c nfc_dev_att_r_q.h nfc_dev_att_r_q.c	/dev/nfc_dev 文件节点， 作为用户态的接口。
comm_driver(dir)	实现主要的驱动功能，不需要改动。			
	board_nfc.h	-		
	priv_nfc_driver.h	-		
	priv_nfc_driver.c	-		

2.3.1 oal_os.h OS 接口

需要适配实现接口，建议放在.h 文件中，通过 inline 或者函数宏实现，具体可以参考已有的代码实现。适配实现接口如表 2-2 所示。

表2-2 oal_os.h 接口函数说明

函数	功能说明
void oal_msleep(uint32_t time)	延时实现
void oal_udelay(uint32_t time)	
void oal_mutex_init(oal_mutex_t *lock)	互斥量实现
void oal_mutex_destroy(oal_mutex_t *lock)	
void oal_mutex_lock(oal_mutex_t *lock)	
oal_mutex_unlock(oal_mutex_t *lock)	
oal_spin_lock_init	临界段实现
oal_spin_lock_irqsave	
oal_spin_unlock_irqrestore	
void oal_init_waitqueue_head(oal_wait_queue_head_t *wq)	唤醒事件实现
int oal_wait_event_interruptible_timeout(oal_wait_queue_head_t	

函数	功能说明
wq, int condition, unsigned long timeout)	
void oal_wkup_event(oal_wait_queue_head_t wq)	

2.3.2 oal_plat_op.h 文件操作和日志打印接口

需要适配实现文件操作和日志打印接口，建议放在.h 文件中，通过 inline 或者函数宏实现，具体可以参考已有的代码实现。接口函数如表 2-3 所示。

表2-3 oal_plat_op.h 接口函数说明

函数	功能说明
oal_file_t *oal_file_open_readonly(const char *path)	只读打开文件。
oal_file_read_ext(oal_file_t *file, oal_loff_t pos, char *buf, uint32_t count)	读文件。
int32_t oal_file_close(oal_file_t *file)	关闭文件。
int oal_file_open_get_len(const char *path, oal_file_t **fp, unsigned int *file_len)	获取文件长度。
PS_PRINT_FUNCTION_NAME	打印当前函数名。
PS_PRINT_DBG(s, args...)	debug 级别打印。
PS_PRINT_INFO(s, args...)	info 级别打印。
PS_PRINT_SUC(s, args...)	打印执行成功。
PS_PRINT_WARNING(s, args...)	warning 级别打印。
PS_PRINT_ERR(s, args...)	error 级别打印。
PS_PRINT_ALERT(s, args...)	alert 级别打印。

2.3.3 board_q.c 板级接口

实现 AP 侧与 NFC 芯片之间的硬件接口操作，如 GPIO 控制、I2C 读写等。NFC 芯片的 I2C 地址是 0x28，时钟最高支持 full speed 400K。需要注意的是，需要在板级接口

中实现“dev_wakeup_host_gpio”中断函数，并注册，中断函数中需要调用“comm_driver”模块中的“nfc_wakeup_host_gpio_isr”函数。具体可以参考已有的代码实现如表 2-4 所示。

表2-4 board_q.c 接口函数说明

函数	功能说明
int board_init(void *param)	板级初始化，主要是 GPIO，数据结构等等的初始化。
void board_deinit(void)	NFC 关闭时的去初始化动作。
void board_pwn_ctrl(nfc_enable_ctrl_t enable)	控制 NFC 芯片上下电。
void board_wkup_host_gpio_int_enable(nfc_enable_ctrl_t en)	dev_wakeup_host_gpio 中断使能控制。
int board_get_dev_wkup_host_status(void)	获取 dev_wakeup_host_gpio 状态。
int board_get_host_wkup_dev_status(void)	获取 host_wakeup_dev_gpio 状态。
void board_set_host_wkup_dev_value(int value)	设置 host_wakeup_dev_gpio 状态。
const char *board_get_fw_cfg_path(void)	获取固件文件存放路径。
int board_i2c_write(char *buf, int count)	通过 I2C 往 NFC 芯片写数据（默认地址 0x28）。
int board_i2c_read(char *buf, int count)	通过 I2C 从 NFC 芯片读数据（默认地址 0x28）。

2.3.4 安全函数库编译指导

- 1. 参照 2.2 获取驱动源码获取安全函数库源码。
- 2. 将如下文件添加到项目中，参加编译。

源文件	文件说明
-----	------

源文件	文件说明
include/securec.h	安全函数对外接口头文件。
include/securectype.h	对外接口辅助头文件，被 securec.h 包含。
src/memcpy_s.c	提供 memcpy_s 函数。
src/memmove_s.c	提供 memmove_s 函数。
src/memset_s.c	提供 memset_s 函数。
src/strcpy_s.c	提供 strcpy_s 函数。
src/strncpy_s.c	提供 strncpy_s 函数。
src/strcat_s.c	提供 strcat_s 函数。
src/strncat_s.c	提供 strncat_s 函数。
src/secureutil.c	提供获取安全函数版本号的函数。
src/snprintf_s.c	提供 snprintf_s 和 snprintf_truncated_s 函数。
src/vsnprintf_s.c	提供 vsnprintf_s 和 vsnprintf_truncated_s 函数。
src/sprintf_s.c	提供 sprintf_s 函数。
src/vsprintf_s.c	提供 vsprintf_s 函数。
src/secureprintoutput_a.c(包含 output.inl)	该文件为格式化输出公共函数，不提供浮点数以及宽字符处理功能。
src/sscanf_s.c	提供 sscanf_s 函数。
src/vsscanf_s.c	提供 vsscanf_s 函数。
src/secureinput_a.c(包含 input.inl)	该文件为格式化输入公共函数，不提供浮点数以及宽字符处理功能。
src/strtok_s.c	提供 strtok_s 函数。

3. 修改 Makefile 适配内核编译

```

MK_FILE_PATH := $(abspath $(lastword $(MAKEFILE_LIST)))
PRIV_KSECUREC_BUILTIN_DIR := $(dir $(MK_FILE_PATH))
PRIV_KSECUREC_INCLUDES := -I$(PRIV_KSECUREC_BUILTIN_DIR)include

```

```

PRIV_KSECUREC_SRC := src

EXTRA_CFLAGS += $(PRIV_KSECUREC_INCLUDES)

ksecurec-objs := memcpy_s.o memmove_s.o memset_s.o  secureutil.o  strcat_s.o
strcpy_s.o strncat_s.o strncpy_s.o sprintf_s.o vsprintf_s.o snprintf_s.o vsnprintf_s.o
secureprintoutput_a.o sscanf_s.o vsscanf_s.o secureinput_a.o
ksecurec-objs := $(addprefix $(PRIV_KSECUREC_SRC)/,$(ksecurec-objs))
obj-y += $(ksecurec-objs)

```

2.3.5 DTS 配置

NFC 驱动需要用到板级 I2C 接口及 GPIO 接口， 需要根据板级硬件连接， 在 DTS 中适对应应的资源， 配置 I2C 速率 full speed 400K， 时钟占空比 40%~60%， 建议 50%。 示例参考如下。 请注意， 示例只是参考， 取决于主芯片平台， 可配置的参数或者格式不一样。

```

&i2c6 {
/* 请根据板级分配的I2C通道号来修改
*/
    pinctrl-names = "default", "sleep";
    pinctrl-0 = <&i2c6_pins_a>;
    pinctrl-1 = <&i2c6_pins_sleep_a>;
    i2c-scl-rising-time-ns = <185>;
    i2c-scl-falling-time-ns = <20>;
    status = "okay";
    nfc@28{
/* 请根据板级分配的I2C地址来修改
*/
        compatible = "nfc_module,nfc";
        status = "ok";
        reg = <0x28>;
        nfc,gpio_wakeup_host = <&gpio1 7 0>;
/* 请根据板级wakeup host GPIO连接情况修改 */
        nfc,gpio_poweron = <&gpio1 4 0>;
/* 请根据板级PWREN GPIO连接情况修改 */
        nfc,gpio_wakeup_device = <&gpio1 6 0>;
/* 请根据板级wakeup device GPIO连接情况修改 */
    };
};

```


2.4 编译内核

在 menuConfig 阶段，选择 Vendor NFC Driver 参与内核编译。

```
Device Drivers ---->
<*> Vendor NFC Driver
```

完成编译后，驱动就在内核映像文件中了。

2.5 测试驱动

在 sys/nfc 目录下存在 NFC 相关的节点，执行以下两条命令可对 nfc_download 进行写入字符 1，然后读取检查 nfc_get_result 是否为 1，从而判断驱动加载是否成功。

```
echo "1" > /sys/nfc/nfc_download      // 芯片上电，加载固件
cat /sys/nfc/nfc_get_result           // 检查驱动加载是否成功，1为成功
```

3

Linux NFC 业务代码

本章节将分步骤讲解如何利用提供的 NFC 源码包移植适配到 Linux 系统。

3.1 步骤一：安装 NFC 源码包

3.2 步骤二：编译 NFC HAL 库文件

3.3 步骤三：集成 NFC 协议栈构建应用

3.4 步骤四：更新 NFC 参数配置文件

3.5 步骤五：NFCC 固件部署

3.6 步骤六：测试 NFC 功能

3.1 步骤一：安装 NFC 源码包

安装 NFC 源码包的方法如下：

- 获取本方案的 NFC 源码包，解压到 Linux 编译环境的某个目录下，该目录名称用 “\$(Linux_nfc_root)” 表示。
- 自行下载获取 “[2.2 获取驱动源码](#)” 介绍的第三方安全函数库 libboundscheck 源码。下载解压后将安全函数库主目录名改成 “secure_c”，放在 “\$(Linux_nfc_root)” 目录下。
- 安装 NFC 源码完成后，\$(Linux_nfc_root)目录下文件夹结构如下：
 - conf：存放配置参数文件的文件夹。
 - firmware：存放 NFC device 固件程序的文件夹。
 - hal：NFC HAL 层主要源码的文件夹。
 - secure_c：安全函数库文件夹。

- Makefile：编译文件。

3.2 步骤二：编译 NFC HAL 库文件

编译 NFC HAL 库文件的步骤如下：

步骤 1 进入 NFC 源码包目录 “\$(Linux_nfc_root)” 下，执行命令 “make” 构建库文件。

默认采用编译环境下的 gcc 和 g++ 进行编译构建。用户可根据自己的目标板系统，采用合适的交叉编译工具链替代 Makefile 中默认的 gcc、g++。

步骤 2 以上命令执行完毕后，会在当前目录生成如下文件夹和文件：

- \$(Linux_nfc_root)/output/obj/：编译生成的 object 文件。
- \$(Linux_nfc_root)/output/lib/：编译生成的静态链接库和动态链接库文件。

----结束

3.3 步骤三：集成 NFC 协议栈构建应用

本方案只提供 NFC HAL 层模块，Linux 系统上的 NFC 应用还需集成用户自有的 NFC 协议栈和上层应用 App 软件，需要一起编译构建，或者是动态链接到 [3.2 步骤二：编译 NFC HAL 库文件](#) 介绍的共享库。无论采用哪一种方案，参考的 NFC HAL 模块的核心接口 API 文件为 “\$(Linux_nfc_root)/hal/hal/hal_main.h”，其他可能涉及的头文件请参见 Makefile 文件中包含的路径。

3.4 步骤四：更新 NFC 参数配置文件

根据 “[5 常见问题](#)” 以及最终产品的实际情况，可选择性地将源码包如下路径的 hal 层配置文件的相关配置参数值进行适当的修改：\$(Linux_nfc_root)/conf/libnfc-hal.conf：包含 HAL 层功能配置参数（详细内容请参见 “[4 NFC 参数配置文件](#)”）。

修改完成后，最终需要将配置文件复制到目标设备文件系统的如下目录（该路径可通过源码的宏定义 NFC_HAL_CFG_FILE 修改）：/vendor/etc/libnfc-hal.conf

3.5 步骤五：NFCC 固件部署

源码包中有 NFCC (NFC Controller) 的固件，可放置到目标设备文件系统如下路径：

- /odm/etc/firmware/RAM.bin
- /odm/etc/firmware/nfc_cfg

请注意，这两个文件需要提前保存在 host 端，占用存储空间一般不超过 150KB，具体视用户所得固件文件大小情况而定。另外，默认拷贝路径可以通过搜索和修改平台驱动代码里的默认路径、以及 nfc_cfg 里的默认路径达成。

3.6 步骤六：测试 NFC 功能

在目标设备上运行 [3.3 步骤三：集成 NFC 协议栈构建应用](#) 构建出来的 NFC App，即可测试 NFC 业务，并通过系统日志查看运行情况。

4 NFC 参数配置文件

“3.4 步骤四：更新 NFC 参数配置文件” 小节中的配置文件，可在 NFC 初始化时配置 HAL 层的相关参数。在将 NFC 集成到新的设备中时，需要注意相关的配置参数是否需要修改。本文仅列举示例参数说明，详细内容和参数注释请参见 NFC 安装包内的示例配置文件。libnfc-hal.conf 文件配置参数如表 4-1 所示。

表4-1 libnfc-hal.conf 文件配置参数表

参数	描述
NFC_DEV_NODE	设备节点名字。
LOG_HAL_LOGLEVEL	HAL 模块的日志等级。
LOG_TML_LOGLEVEL	传输映射层模块的日志等级。
LOG_NCIX_LOGLEVEL	NCI 发送的日志等级。
LOG_NCIR_LOGLEVEL	NCI 接收的日志等级。
FORCE_FW_DNLD	开 NFC 的时候强制上下电和加载 FW。
FORCE_DEV_PWROFF	关 NFC 的时候强制下电。
SET_RF_PARAM	RF 定制化参数配置的载荷内容。

5 常见问题

5.1 驱动打开失败

5.2 配置文件打开失败

5.3 共享库打开失败

5.1 驱动打开失败

在驱动加载有问题或者设备节点权限有问题时，日志中出现如下信息：

```
Open dev node failed!
```

此时可查看设备节点（默认是/dev/nfc_dev）的权限信息，确保 NFC 具有读写权限。
如果权限没问题时，可查看内核日志，检查驱动加载过程中的具体错误信息。

5.2 配置文件打开失败

在配置文件打开失败时，日志中出现如下信息：

```
Cfg file parse failed: XXXX
```

此时需要查看相关的配置文件是否放置在目标设备的准确路径下。

5.3 共享库打开失败

如果将 NFC HAL 编译成共享库使用时，若应用在运行时发生链接错误，需要检查相应的共享库文件是否放置在目标设备的准确路径下，该路径是否已经配置加入 LD_LIBRARY_PATH，以及当时编译共享库的流程和结果是否正确等。

A 缩略语

H

HAL	Hardware Abstract Layer	硬件抽象层
-----	-------------------------	-------

N

NCI	NFC Controller Interface	NFC 控制器接口
NFC	Near Field Communication	近场通信
NFCC	NFC Controller	NFC 控制器